

Homework 01

Kristina

March 30, 2026

2.7

(1)

Algorithm 1 MainValue(A)

Input: 数组 $A[1..n]$

Output: 数组 A 中是否含有主元素

```

1: QUICKSORT( $A, 1, n$ )
2:  $k \leftarrow \lfloor n/2 \rfloor + 1$ 
3: for  $i = 1$  to  $n - k$  do
4:   if  $A[i] = A[i + k]$  then
5:     return 1
6:   end if
7: end for
8: return 0

```

算法时间包括快速排序时间和逐项比较查找主元素的时间

$$T(n) = O(n \log n) + O(n) = O(n \log n)$$

(2)

算法步骤：

- 找出数组 A 中的中位数
- 逐项比较数组 A 中的元素是否与中位数相等，并记录相等的个数

由于数组中的主元素必为中位数，则可以通过中位数出现次数是否 $\geq \lfloor n/2 \rfloor + 1$ 判断
 第一步可以通过最坏情况下 $O(n)$ 的算法实现，第二步时间复杂度 $O(n)$ ，算法总时间 $O(n)$

(3)

模仿芯片测试的方法：

- 将数组 A 复制进数组 B
- 将 A 中元素分为两两一组
- 对于每组中的元素，若二者相等则任取一个保留，否则全部删除
- 重复进行以上操作直到 A 中元素个数 ≤ 1
- 若 A 中无元素，则原数组无主元素
- 若 A 中有一个元素，将 B 中元素与该元素逐项比较并记录相等的元素个数，若 $\geq \lfloor n/2 \rfloor + 1$ ，则有主元素，否则没有主元素

算法时间复杂度：

$$\begin{cases} W(n) = W(n/2) + O(n) \\ W(1) = 0 \end{cases}$$

由主定理， $W(n) = O(n)$

2.12

Algorithm 2 Intersection(A, B)

Input: 不重复的数组 $A[1..n], B[1..m]$

Output: 数组 $C = A \cap B$

```

1:  $k \leftarrow 0$ 
2: QUICKSORT( $B, 1, m$ )
3: for  $i = 1$  to  $n$  do
4:   if BINARYSEARCH( $B, A[i]$ ) then
5:      $C[k + 1] \leftarrow A[i]$ 
6:      $k \leftarrow k + 1$ 
7:   end if
8: end for

```

算法时间包括数组 B 快速排序时间和数组 A 逐项二分查找时间

$$T(n) = O(m \log m) + O(n \log m) = O(n \log m) = O(n \log \log n)$$

2.15

(1)

算法 A:

Findmax 函数调用时间复杂度 $O(n)$ ，直接返回该项下标删除前无需额外查找，执行 i 次最坏情况下 $W(n) = O(in) = O(n^{3/2})$

算法 B:

对 S 排序时间复杂度 $O(n \log n)$, 输出最大 i 个数时间复杂度 $O(i)$, 最坏情况下 $W(n) = O(n \log n) + O(i) = O(n \log n)$

(2)

Algorithm 3 FindIMax(S, i)

Input: 不同数的数组 $S[1..n]$, 正整数 i

Output: 从大到小 S 中最大的 i 项

```

1:  $k \leftarrow n - i + 1$ 
2:  $x \leftarrow \text{SELECT}(S, k)$ 
3:  $k \leftarrow 0$ 
4: for  $j = 1$  to  $n$  do
5:   if  $S[j] \geq x$  then
6:      $A[k + 1] \leftarrow S[j]$ 
7:      $k \leftarrow k + 1$ 
8:   end if
9: end for
10:  $\text{QUICKSORT}(A, 1, k)$ 
11: for  $j = 0$  to  $i - 1$  do
12:   output  $A[i - j]$ 
13: end for
```

算法时间包括找到第 k 大的元素 (调用 Select 函数)、循环查找大于等于该元素的数、排序前 i 大的数、从大到小输出以比较运算作为基本运算, 则最坏情况下的算法时间复杂度为

$$W(n) = O(n) + O(n) + O(i \log i) = O(n) + O(n) + O(n^{1/2} \cdot 1/2 \cdot \log n) = O(n)$$

2.18

(见下页 Algorithm 4)

算法时间包括循环计算到原点的距离、找到第 $\lfloor \sqrt{n} \rfloor$ 远的距离 (调用 Select 函数)、循环查找大于等于该元素的数、排序前 $\lfloor \sqrt{n} \rfloor$ 大的数、从大到小输出标号以比较运算作为基本运算, 则最坏情况下的算法时间复杂度为

$$W(n) = O(n) + O(n) + O(n^{1/2} \log n^{1/2}) = O(n)$$

3.3

使用如下动态规划算法, 其中 $dp[i][j]$ 或 $A[j]$ 中记录当前情况下使用前 i 个货柜占用不超过 j 长度能得到的最大收益

Algorithm 4 FindPoint(x,y)**Input:** 横坐标数组 $x[1..n]$, 纵坐标数组 $y[1..n]$ **Output:** 从大到小距离原点最远的 $\lfloor \sqrt{n} \rfloor$ 个点的标号

```

1: for  $i = 1$  to  $n$  do
2:    $d[i] \leftarrow \sqrt{x[i]^2 + y[i]^2}$ 
3: end for
4:  $k \leftarrow \lfloor \sqrt{n} \rfloor$ 
5:  $a \leftarrow \text{SELECT}(d, n - k + 1)$ 
6:  $j \leftarrow 0$ 
7: for  $i = 1$  to  $n$  do
8:   if  $d[i] \geq a$  then
9:      $A[j + 1] \leftarrow (d[i], i)$ 
10:     $j \leftarrow j + 1$ 
11:   end if
12: end for
13:  $\text{QUICKSORT}(A, 1, j)$  // 按照  $d[i]$  大小对  $A$  排序
14: for  $i = 0$  to  $k - 1$  do
15:   output  $A[k - i]$  的第二项
16: end for

```

当从使用前 $i - 1$ 个货柜变为使用前 i 个时:

$$dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - l_i] + v_i)$$

前项表示不加入第 i 个, 后项表示加入第 i 个

考虑到算法最内层循环中 j 从大到小迭代, 可以通过设置 $j \geq L[i]$ 为边界条件来省略判断是否仍有放入第 i 个货柜的空间; 另外每个 $dp[i - 1][j]$ 和 $dp[i - 1][j - L[i]]$ 只会分别在下一个 i 的循环中使用一次, 故可以直接替代, 使用一维数组而非二维数组记录中间结果

Algorithm 5 Cabinet(L, V, D)**Input:** 正整数数组 $L[1..n], V[1..n]$, 正整数 D **Output:** 最大收益 $C[D]$

```

1: for  $i = 1$  to  $n$  do
2:   for  $j = D$  downto  $L[i]$  do
3:      $A[j] \leftarrow \max(A[j], A[j - L[i]] + V[i])$ 
4:   end for
5: end for
6: return  $A[D]$ 

```

算法时间由两重循环构成, 循环内部可在常数时间内完成, 最坏情况下的时间复杂度为 $O(nD)$

3.8

原问题等价于求解

$$\max \sum_{j=1}^n a_j x_j$$

$$\begin{cases} \sum_{j=1}^n a_j x_j \leq N/2 \\ x_j \in \{0, 1\} \end{cases}$$

当最优化的目标函数值为 $N/2$ 时, 说明集合 A 能够被划分为 A_1 和 A_2 , 否则不能使用 0-1 背包问题的求解算法, 最坏情况下的时间复杂度为 $O(nN)$

3.10

$$\max \sum_{j=1}^n v_j x_j$$

$$\begin{cases} \sum_{j=1}^n w_j x_j \leq W \\ \sum_{j=1}^n c_j x_j \leq V \\ x_j \in \{0, 1\} \end{cases}$$

使用如下动态规划算法, 其中 $dp[i][j][k]$ 中记录当前情况下使用前 i 种物品占用不超过 j 重量 k 体积能达到的最大价值

当从使用前 $i-1$ 种物品变为使用前 i 个时:

$$dp[i][j][k] = \max(dp[i-1][j][k], dp[i-1][j-w_i][k-c_i] + v_i)$$

算法步骤:

- i 从 1 到 n 递增
- j 从 W 到 w_i 递减
- k 从 V 到 c_i 递减
- 最内层循环内更新 $A[j][k]$ 记录使用当前前 i 种物品时能达到的最大价值
- 循环结束后, 返回 $A[W][V]$ 作为结果

最坏情况下的时间复杂度为 $O(nWV)$

3.17

- 标记第一行的数累计和 $A[1][1] = 0$ ，此时的最短路径为空
- 对于第 k 层中的 k 个数，逐个计算 $A[k][i] = \min(A[k-1][i-1] + a_{(k-1)(i-1)}, A[k-1][i] + a_{(k-1)i})$ （不存在的部分记为 0），并按照更小的值在前继点上延长记录到达该点的最短路径
- 计算完到达第 n 行每个点的最短路径长度之后，比较取出其中的最小值及其对应的一条路径

算法时间复杂度为 $O(n^2)$